

Lập trình hướng đối tượng Collection và Generic trong Java

GV: ThS. Ngô Tiến Đức

Nội dung chính

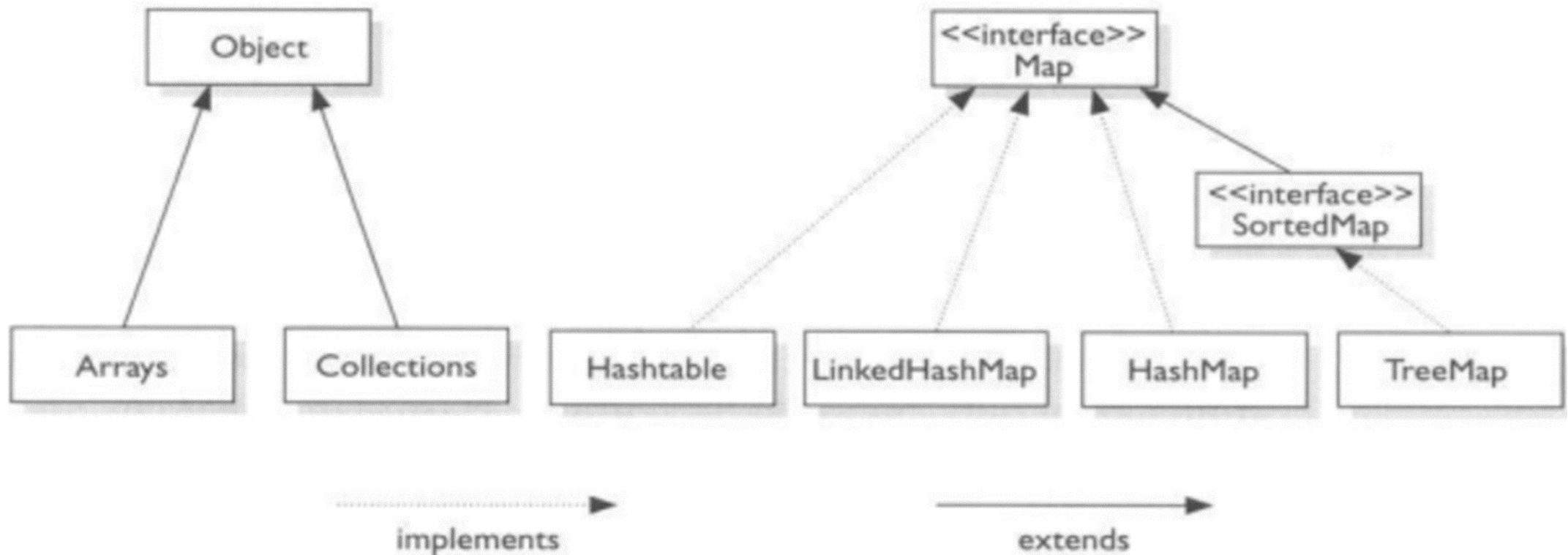
- Java Collections Framework
- Collection
- ArrayList
- Lập trình Generic
- Sắp xếp ArrayList
- Các thành phần khác của Collection
- Map



3

Java Collections Framework

Các thành phần trong Java Collections Framework:



Collection

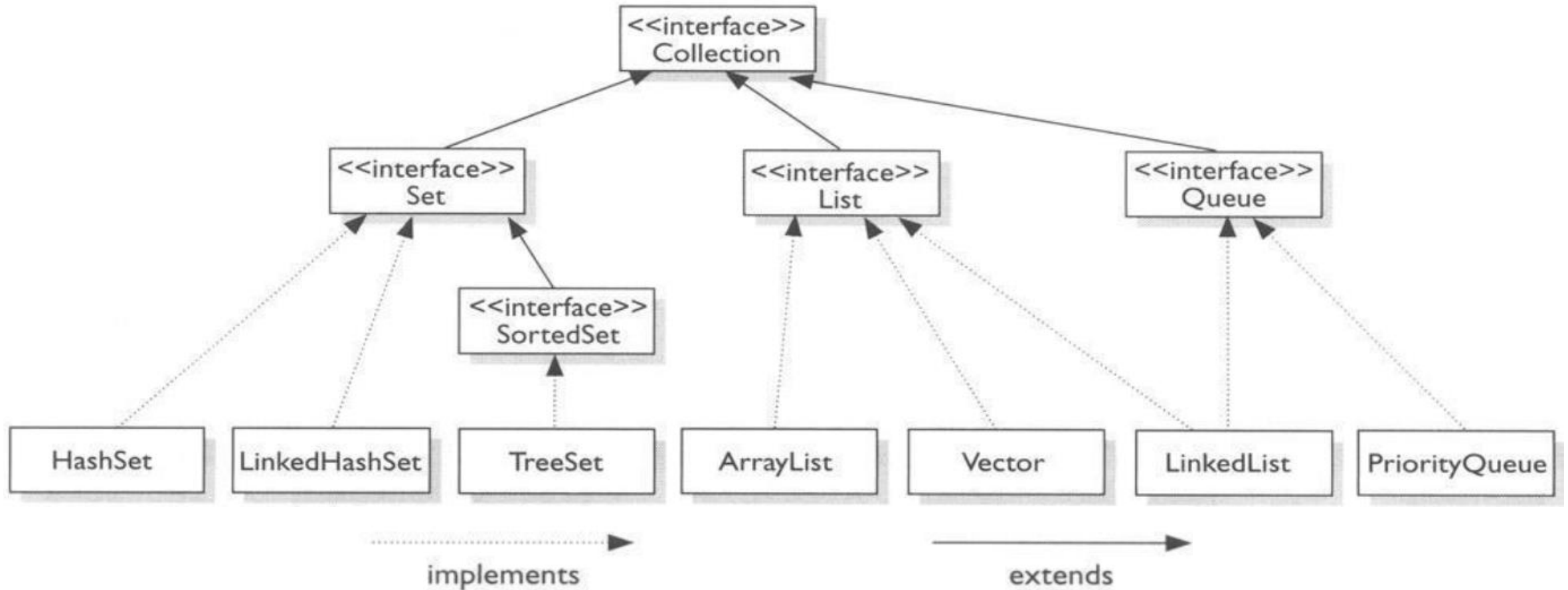
Collection: Lưu trữ và thao tác một tập hợp các đối tượng

- Hỗ trợ nhiều hàm built-in: Chỉ cần khai báo và gọi những phương thức đã được định nghĩa
- Tương tự mảng (Array) nhưng linh hoạt hơn

Array	Collection
Kích thước cố định	Kích thước linh hoạt
Chứa các phần tử có cùng kiểu dữ liệu nguyên thủy hoặc đối tượng	Chứa các phần tử có kiểu dữ liệu đối tượng
Phải duyệt các phần tử thông qua index	Có thể duyệt các phần tử thông qua index hoặc giá trị của phần tử

Collection

Các thành phần trong Collection:



Các phương thức thao tác với Collection:

- Kiểm tra số phần tử: `isEmpty`, `size`
- Thêm/xóa đối tượng: `add`, `remove`
- Kiểm tra đối tượng nằm trong collection hay không: `contains`
- Duyệt các phần tử: `iterator`
- Thao tác số lượng lớn trên toàn bộ collection: `containsAll`, `addAll`,
`removeAll`, `clear`
- ...

ArrayList

ArrayList là phiên bản thông minh hơn của Array: Mảng động có thể thay đổi kích thước

- Cách truy xuất phần tử giống với Array
- Các phần tử có kiểu Object
- Khởi tạo:

```
ArrayList al = new ArrayList(); // trống
```

```
ArrayList al = new ArrayList(c); // khởi tạo từ Collection c
```

```
ArrayList al = new ArrayList(10); // capacity
```

```
List al = new ArrayList(); // đa hình
```


ArrayList

Các phương thức built-in của ArrayList:

Phương thức	Mô tả
<code>void add(int index, Object o)</code>	Chèn phần tử o vào vị trí index
<code>boolean add(Object o)</code>	Thêm phần tử o vào cuối list
<code>boolean addAll(Collection c)</code>	Thêm các phần tử trong Collection c vào cuối list
<code>void clear()</code>	Xóa tất cả phần tử trong list
<code>boolean contains(Object o)</code>	Kiểm tra list có chứa phần tử o không
<code>Object get(int index)</code>	Trả về phần tử tại vị trí index

ArrayList

Các phương thức built-in của ArrayList:

Phương thức	Mô tả
<code>Object set(int index, Object o)</code>	Thay thế phần tử tại vị trí index bởi o
<code>int size()</code>	Trả về số phần tử trong list
<code>int indexOf(Object o)</code>	Trả về index đầu tiên của phần tử o trong list (-1 nếu không có trong list)
<code>int lastIndexOf(Object o)</code>	Trả về index cuối cùng của phần tử o trong list
<code>Object remove(int index)</code>	Xóa phần tử tại vị trí index
<code>boolean remove(Object o)</code>	Xóa phần tử o đầu tiên trong list

ArrayList

Duyệt phần tử trong ArrayList:

```
for(int i = 0; i < al.size(); i++) {  
    System.out.println(al.get(i));  
}
```

```
for(Object o : al) {  
    System.out.println(o);  
}
```

```
al.forEach(o -> {  
    System.out.println(o);  
});
```

ArrayList

Vấn đề khi loại bỏ các phần tử trong quá trình duyệt:

```
ArrayList al = new ArrayList();  
al.add("A");  
al.add("B");  
al.add("C");  
for(Object o : al){  
    al.remove(o); // ConcurrentModificationException  
}  
// tương tự với forEach
```

ArrayList

Duyệt phần tử trong ArrayList với Iterator:

- Iterator là một interface cho phép duyệt các phần tử trong collection
- Khởi tạo: `Iterator <name> = <ArrayList_name>.iterator();`
- Các phương thức của Iterator:
 - `boolean hasNext()`: Kiểm tra xem còn phần tử nào để duyệt không
 - `Object next()`: Trả về phần tử tiếp theo trong collection
 - `void remove()`: Xóa phần tử cuối cùng được trả về từ `next()`

ArrayList

Duyệt phần tử trong ArrayList với Iterator:

- Sử dụng remove của iterator là cách an toàn để xóa các phần tử khỏi Collection trong khi duyệt

```
Iterator iterator = al.iterator();
while(iterator.hasNext()) {
    Object o = iterator.next();
    if(o.equals("B")) iterator.remove();
}
```

- Xóa phần tử khi duyệt theo size() có an toàn không?

Lập trình Generic

- Khi khai báo một lớp của Collections Framework, các phần tử sẽ có kiểu mặc định là **Object**

- VD:

```
ArrayList al = new ArrayList();
```

```
al.add("abc"); // ok
```

```
al.add(1); // ok
```

-> Khó kiểm soát về kiểu dữ liệu và khi thực hiện ép kiểu

- Vậy làm sao để các phần tử trong tập hợp chỉ có một kiểu dữ liệu?

Lập trình Generic

- Generic: Các thành phần của Collections Framework có thể cố định kiểu dữ liệu cho các phần tử bên trong đặt trong dấu <>

- VD:

```
ArrayList<String> al = new ArrayList<>();
```

```
al.add("abc"); // ok
```

```
al.add(1); // lỗi
```

- Vậy làm sao để viết một phương thức chung để thao tác trên các tập hợp có kiểu dữ liệu khác nhau?

Lập trình Generic

- Tham số hóa kiểu dữ liệu: Cho phép tạo ra một lớp hoặc interface hoặc phương thức để thao tác với nhiều kiểu dữ liệu khác nhau

```
public static <T> void printArrayList(ArrayList<T> al) {  
    for(T element: al) {  
        System.out.print(element + " ");  
    }  
    System.out.println();  
}
```

Lập trình Generic

```
public static void main(String[] args) {  
    ArrayList<String> alStr = new ArrayList<>(Arrays.asList("a",  
        "cd", "bbb"));  
    ArrayList<Integer> alInt = new ArrayList<>(Arrays.asList(1,  
        2, 3));  
    printArrayList(alStr);  
    printArrayList(alInt);  
}
```

Lập trình Generic

Các ký hiệu thường dùng trong lập trình Generic

Ký hiệu	Ý nghĩa
E	Element – Phần tử
K	Key – Khóa
V	Value – Giá trị
T	Type – Kiểu dữ liệu
N	Number – Số

Lập trình Generic

```
public class Pair<K, V> {  
    private K key;  
    private V value;  
    ... // constructors, getters, setters, toString  
  
    public static void main(String[] args) {  
        Pair<Integer, Integer> p1 = new Pair<>(1, 100);  
        Pair<Integer, String> p2 = new Pair<>(2, "ABC");  
        ...  
    }  
}
```

Sắp xếp ArrayList

Sắp xếp các đối tượng trong ArrayList theo thuộc tính:

- Ví dụ: Lớp "sinh viên" có thuộc tính "tên" và "điểm", hãy sắp xếp danh sách sinh viên theo thứ tự điểm tăng dần

```
public class Student {  
    String name;  
    int score;  
    ... // constructors, getters, setters, toString  
}
```

- Arrays.sort()?

Sắp xếp ArrayList

```
public static void main(String[] args) {  
    Student s1 = new Student("Duy", 10);  
    Student s2 = new Student("Nam", 8);  
    ...  
    ArrayList<Student> studentList = new ArrayList();  
    studentList.add(s1);  
    studentList.add(s2);  
    ...  
    /* có thể thêm đồng thời nhiều phần tử sử dụng Arrays.asList  
       hoặc Collections.addAll */  
}
```

Sắp xếp ArrayList

Sắp xếp với Comparable:

- Cho class Student kế thừa interface Comparable

```
public class Student implements Comparable<Student> {  
    ...  
    @Override  
    public int compareTo(Student o) {  
        return (this.getScore() - o.getScore());  
    }  
}
```

Sắp xếp ArrayList

Sắp xếp với Comparable:

- Sử dụng `Collections.sort()` để sắp xếp

```
public static void main(String[] args) {  
    ...  
    Collections.sort(studentList);  
    System.out.println(studentList.toString());  
}
```


Sắp xếp ArrayList

Sắp xếp với Comparator:

- Cách 1: Khai báo lớp sắp xếp kế thừa interface Comparator

```
public class SortByScore implements Comparator<Student> {  
    @Override  
    public int compare(Student sv1, Student sv2) {  
        return (sv1.getScore() - sv2.getScore());  
    }  
    // cùng class với main thì để static  
}
```

Sắp xếp ArrayList

Sắp xếp với Comparator:

- Cách 1: Khai báo lớp sắp xếp kế thừa interface Comparator

```
public static void main(String[] args) {  
    ...  
    studentList.sort(new SortByScore());  
    // Collections.sort(studentList, new SortByScore());  
    System.out.println(studentList.toString());  
}
```

Sắp xếp ArrayList

Sắp xếp với Comparator:

- Cách 2: Khai báo Comparator dưới dạng "Anonymous Class"

```
studentList.sort(new Comparator<Student>() {  
    @Override  
    public int compare(Student sv1, Student sv2) {  
        return (sv1.getScore() - sv2.getScore());  
    }  
});  
  
System.out.println(studentList.toString());
```

Sắp xếp ArrayList

Từ Java 8: Có thể sử dụng **lambda expression** với hàm so sánh tự định nghĩa tiêu chí:

```
studentList.sort((Student sv1, Student sv2) -> {  
    return (sv1.getScore() - sv2.getScore());  
});
```

Nếu bên trong hàm chỉ có return, có thể viết ngắn gọn:

```
studentList.sort((Student sv1, Student sv2) ->  
    (sv1.getScore() - sv2.getScore()));
```

Sắp xếp ArrayList

Từ Java 8: Có thể sử dụng **lambda expression** với hàm so sánh mặc định:

```
studentList.sort(Comparator.comparing((Student s) ->  
s.getScore())); // sắp xếp mặc định tăng dần
```

```
studentList.sort(Comparator.comparing((Student s) ->  
s.getScore()).reversed()); // đảo ngược thứ tự
```

Luyện tập

Lớp "sinh viên" có thuộc tính "tên" và 3 thuộc tính liên quan đến điểm của 3 môn (kiểu int), hãy sắp xếp danh sách sinh viên theo thứ tự:

- Điểm trung bình tăng dần
- Tên theo alphabet
- Điểm trung bình tăng dần, nếu điểm bằng nhau thì sắp xếp theo tên alphabet

Các thành phần khác của Collection

Interface **List**: Collection có thứ tự và có thể chứa các phần tử giống nhau

- **Vector**: Tương tự ArrayList nhưng có khả năng đồng bộ hóa khi chạy đa luồng
- **LinkedList**: Cấu trúc dữ liệu bao gồm một nhóm các nút (node) tạo thành một chuỗi, dữ liệu ở mỗi nút tham chiếu đến nút kế tiếp trong chuỗi

Các thành phần khác của Collection

Interface **Queue**: Tương tự như List nhưng hoạt động theo nguyên tắc hàng đợi First-In-First-Out, chỉ truy cập phần tử đầu hoặc cuối, các phần tử được thêm vào cuối và xóa từ đầu

- **PriorityQueue**: Cấu trúc hàng đợi ưu tiên, các phần tử được sắp xếp (mặc định hoặc tùy chỉnh với Comparator), truy cập phần tử có "độ ưu tiên" cao nhất, không chứa phần tử null
- **LinkedList**: Cho phép thêm và xóa phần tử ở cả đầu và cuối hàng đợi, có thể chứa phần tử null

Các thành phần khác của Collection

Interface **Set**: Collection không chứa các phần tử trùng nhau

- **HashSet**: Sử dụng bảng băm (hash table) để lưu trữ phần tử, là lớp thực thi có tốc độ tốt nhất nhưng không đảm bảo thứ tự của phần tử được đưa vào
- **LinkedHashSet**: Cũng sử dụng bảng băm nhưng các phần tử được sắp xếp theo thứ tự đưa vào
- **TreeSet**: Tự động sắp xếp thứ tự phần tử theo giá trị

Các thành phần khác của Collection

```
Integer[] arr = { 20, 1, 4, 2, 1, 3 };  
ArrayList<Integer> al = new ArrayList<>(Arrays.asList(arr));  
Set hashSet = new HashSet(al);  
System.out.println(hashSet); // [1, 2, 3, 20, 4]  
Set linkedHashSet = new LinkedHashSet(al);  
System.out.println(linkedHashSet); // [20, 1, 4, 2, 3]  
Set treeSet = new TreeSet(al);  
System.out.println(treeSet); // [1, 2, 3, 4, 20]
```

Map

Interface **Map**: Lưu trữ các phần tử dưới dạng **Key – Value (K – V)**:

- Mỗi **key** ánh xạ tới một **value** -> các **key** không thể trùng nhau
- **Hashtable** là một lớp cũ hỗ trợ đồng bộ hóa và không còn được sử dụng thường xuyên
- Các lớp **HashMap**, **LinkedHashMap**, **TreeMap** hoạt động tương tự các lớp của **Set**

Map

Các phương thức built-in của Map:

Phương thức	Mô tả
<code>V put(K key, V value)</code>	Thêm một cặp key-value vào Map (nếu key đã tồn tại, value cũ sẽ được thay thế)
<code>V get(Object key)</code>	Trả về value tương ứng với key
<code>V remove(Object key)</code>	Loại bỏ cặp key-value tương ứng với key
<code>boolean containsKey(Object key)</code>	Kiểm tra xem Map có chứa khóa key hay không
<code>boolean containsValue(Object value)</code>	Kiểm tra xem Map có chứa giá trị value hay không

Map

Các phương thức built-in của Map:

Phương thức	Mô tả
<code>Set<K> keySet()</code>	Trả về Set chứa tất cả các key trong Map
<code>Collection<V> values()</code>	Trả về Collection chứa tất cả các value trong Map
<code>Set<Map.Entry<K, V>> entrySet()</code>	Trả về Set chứa tất cả các cặp key-value trong Map
<code>void clear()</code>	Xóa tất cả phần tử trong Map
<code>int size()</code>	Trả về số lượng cặp key-value trong Map
<code>boolean isEmpty()</code>	Kiểm tra xem Map có rỗng hay không

Map

```
public static void main(String[] args) {  
    Map<Integer, String> map = new HashMap<>();  
    map.put(1, "A");  
    map.put(1, "A1");  
    map.put(2, "B");  
    map.put(3, "C");  
    System.out.println(map); // [A1, B, C]  
}
```

Map

VD: Duyệt các phần tử trong Map:

```
// lấy ra tất cả key
for(<key_type> key : <map_name>.keyset()) {...}

// lấy ra tất cả value
for(<value_type> value : <map_name>.values()) {...}

// lấy ra tất cả key - value
for(Map.Entry<<key_type>,<value_type>> entry : <map_name>.entrySet())
{...}
```

Luyện tập

Nhập vào một xâu ký tự không quá 500 từ và không chứa ký tự đặc biệt, hãy đếm số lần xuất hiện của từng từ trong xâu. Viết 2 chương trình:

- Chương trình không sử dụng Collections Framework
- Chương trình có sử dụng Collections Framework

Tài liệu tham khảo

- Liang, Y. Daniel, Introduction to Java programming and Data Structures, 11th edition, Pearson Prentice Hall, 2019
- N. M. Sơn, Bài giảng Lập trình hướng đối tượng, HVCNBCVT, 2020
- N. M. Sơn, Slide giảng dạy môn Lập trình hướng đối tượng
- T. T. V. Anh, Slide giảng dạy môn Lập trình hướng đối tượng